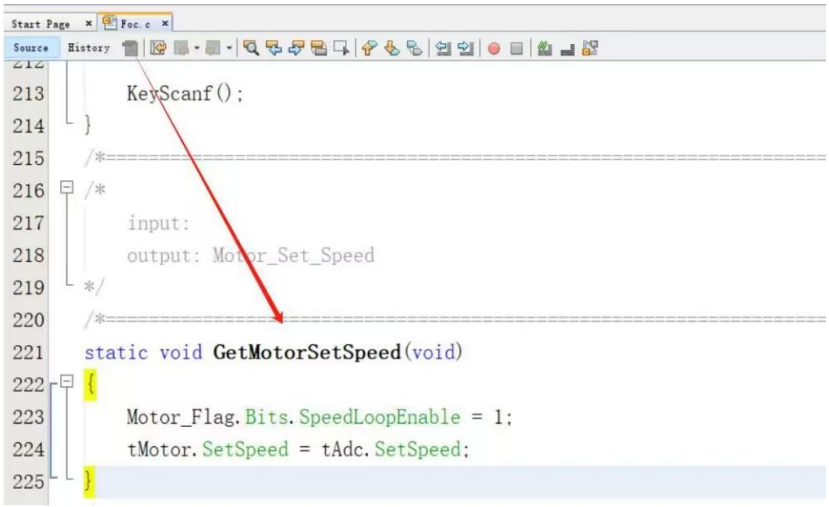
76-FOC 29

大家晚上好，欢迎参加张飞实战电子 FOC 第 11 期学习。

上次课呢，我们把 C 语言写的 PID 函数讲完了。那么，今天呢，我们就主要来分析下速度环相关的代码。我们要进行速度控制的话，需要一个目标转速，还需要反馈转速。目标转速呢，可以通过板上的电位器来设定。

而估算转速（反馈转速），是来自于电流观测器。所以，速度控制需要的两个速度量我们都获取到了，求取出这两个量的差值，做一个 PID 控制，就可以实现对转速的控制。我们编写代码的时候呢，也是需要把反馈转速，设定转速给获取到，最后再调用编写的 PID 函数，来实现转速控制。



大家看下，这个函数是获取设定转速，设定转速呢，是来自于 Adc 采样得到的旋钮值。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
97   /*=============================================================
98   void AdcPotProcess(void)
99   {
00       unsigned int Temp;
01       UNION_U32 unTemp32;
02
03       Temp = tAdc.An4.ValueAverage << 6;
04
05       if (Temp < 200)                //MOTOR_MIN_SPEED
06           Temp = 0;
07
08       unTemp32.U32 = __builtin_muluu(2500,Temp);
09       Temp = unTemp32.Words.High;
10
11       if (Temp > 2500)
12           Temp = 2500;
13
14       tAdc.SetSpeed = Temp;
15   }
16   /*=============================================================
```

因为我们电机转速是 2000RPM，所以，这里先对电位器采集值　做了处理，由 0~1023 变到　0~65535。然后呢，再乘以 2500，取高 16 位，这样的话，tAdc.SetSpeed 这个值的范围就是　0~2500 了。这样处理之后呢，可以作为转速设定值，所以，拧旋钮，就可以调节电机的转速。

我们继续往下。

```
/*=============================================================*/
void AdcPotProcess(void)
{
    unsigned int Temp;
    UNION_U32 unTemp32;

    Temp = tAdc.An4.ValueAverage << 6;

    if (Temp < 200)              //MOTOR_MIN_SPEED
        Temp = 0;

    unTemp32.U32 = __builtin_muluu(2500,Temp);
    Temp = unTemp32.Words.High;

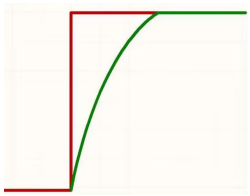
    if (Temp > 2500)
        Temp = 2500;

    tAdc.SetSpeed = Temp;
}
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
    output: Motor_Set_Speed
*/
/*==========================================================*/
static void GetMotorSetSpeed(void)
{
    Motor_Flag.Bits.SpeedLoopEnable = 1;
    tMotor.SetSpeed = tAdc.SetSpeed;
}
/*==========================================================*/
```

那么，刚才处理好旋钮的值之后，就在这个地方进行赋值。但是呢，有的时候这个设定转速不是直接就作为速度环的目标值，而是希望目标转速缓慢变化。换句话说，如果旋钮拧得太快，电机也会跟不上调节，所以目标转速需要有加减速控制。因此呢，速度环的目标转速就会按照一定的斜坡递增到设定转速，或者递减到设定转速。



大家看下这幅图，红色的假设是我们的给定转速，那么，这个绿色的就是速度环的目标转速。这样做的目的呢，就是想让速度调节变得柔和一些。那么，这个设定转速怎么转换为这个目标转速呢？ 我们下面来看下对应的代码。

```
/*==========================================================*/
static void GetMotorAimSpeed(void)
{
    unsigned int Temp16;

    if (tMotor.SetSpeed ==0)
    {
        tMotor.AimSpeed = 0;
    }
    else
    {
        if (!Motor_Flag.Bits.SpeedLoopEnable)
        {
            tMotor.AimSpeed = 300;
        }
        else
        {
            DISIE();
            Temp16 = tFoc.SpeedCount;
            ENIE();
```

这里呢，把获取期望转速封装为了一个函数。如果说，设定转速是 0 的话，期望转速也直接就是 0，不用做什么斜坡。如果设定转速不是 0 了，就会进入到 else 分支。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    if (!Motor_Flag.Bits.SpeedLoopEnable)
    {
        tMotor.AimSpeed = 300;
    }
    else
    {
        DISIE();
        Temp16 = tFoc.SpeedCount;
        ENIE();

        if (Temp16 == 0)
        {
            DISIE();
            tFoc.SpeedCount = MOTOR_AIM_TIME;          //200ms
            ENIE();

            if (tMotor.SetSpeed > tMotor.AimSpeed)
```

这里呢，根据速度环使能标志位进行了判断，如果标志为 0，则给定期望转速是最小转速。如果速度环使能标志是 1，则进入下面的 else 分支。实际情况呢，这个标志为我们给的始终是 1，所以必定会进入 else 分支。有的情况呢，可能会不需要转速环，只需要电流环，所以这个标志位可以作为切换环路工作状态。这里呢，我们默认是双环串级工作，所以呢，这个标志位这里就直接设定为 1 了。

```
else
{
    DISIE();
    Temp16 = tFoc.SpeedCount;
    ENIE();

    if (Temp16 == 0)
    {
        DISIE();
        tFoc.SpeedCount = MOTOR_AIM_TIME;          //200ms
        ENIE();

        if (tMotor.SetSpeed > tMotor.AimSpeed)
```

然后，如果速度环标志位使能的话，就会进入到这个 else 分支，这里呢，是一个时间控制，也就是目标转速的变化，是依据这个时间周期来完成的。也就是说，每隔一个时间，来让目标转速变化一次。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    DISIE();
    Temp16 = tFoc.SpeedCount;
    ENIE();

    if (Temp16 == 0)
    {
        DISIE();
        tFoc.SpeedCount = MOTOR_AIM_TIME;          //200ms
        ENIE();

        if (tMotor.SetSpeed > tMotor.AimSpeed)
        {
            Temp16 = tMotor.SetSpeed - tMotor.AimSpeed;
            if (Temp16 > INC_SPEED_RPM)
                tMotor.AimSpeed += INC_SPEED_RPM;
            else
                tMotor.AimSpeed = tMotor.SetSpeed;
```

例如，我们设定 200ms 作为一个时间周期。让目标转速 200ms 更新一次，往设定转速这个值去靠近。因为这个变量是中断里面递减的，所以，递减到 0 的话，就说明我们设定的时间周期到了。

```
if (Temp16 == 0)
{
    DISIE();
    tFoc.SpeedCount = MOTOR_AIM_TIME;          //200ms
    ENIE();

    if (tMotor.SetSpeed > tMotor.AimSpeed)
    {
        Temp16 = tMotor.SetSpeed - tMotor.AimSpeed;
        if (Temp16 > INC_SPEED_RPM)
            tMotor.AimSpeed += INC_SPEED_RPM;
        else
            tMotor.AimSpeed = tMotor.SetSpeed;
    }
```

到了这里的话，就会进入到这个 if 分支。然后呢，要对计数变量重新设置一个初值。这是为了下一次更新目标转速，接着呢，就来处理目标转速。

```
if (tMotor.SetSpeed > tMotor.AimSpeed)
{
    Temp16 = tMotor.SetSpeed - tMotor.AimSpeed;
    if (Temp16 > INC_SPEED_RPM)
        tMotor.AimSpeed += INC_SPEED_RPM;
    else
        tMotor.AimSpeed = tMotor.SetSpeed;
}
```

大家看下，如果说设定转速大于目标转速，那么目标转速就是递增变到设定转速。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这里的方法呢，就是先求出两者的差值。如果差值大于设定的阈值，则目标转速就按照这个阈值递增到设定转速。如果差值小于设定的阈值，则目标转速就直接被赋值为设定转速。

```
if (tMotor.SetSpeed < tMotor.AimSpeed)
{
    Temp16 = tMotor.AimSpeed - tMotor.SetSpeed;
    if (Temp16 > DEC_SPEED_RPM)
        tMotor.AimSpeed -= DEC_SPEED_RPM;
    else
        tMotor.AimSpeed = tMotor.SetSpeed;
}
```

同理呢，如果设定转速小于目标转速，则目标转速要递减靠近设定值。这里，同样是要把两者的差值求解出来。

如果差值大于递减的阈值，则目标转速就按照这个阈值递减。如果说这个差值小于递减的阈值，则目标转速直接赋值为设定转速。

```
#define INC_SPEED_RPM       100
#define DEC_SPEED_RPM       100
```

两个阈值呢，是宏定义设定的。当然，两个值可以不一样，这样的话，递增跟递减的斜率可以不一样。这两个值的大小呢，可以取决于转速递减或递减的需要的快慢。上面目标转速的获取代码，大家看下，可以理解吧？

```
/*=============================================================*/
//input:    tObs.Erpm
static void GetMotorActSpeed(void)
{
    unsigned int Temp;

    DISIE();
    Temp = tObs.Erpm;
    Motor_Flag.Bits.SpeedLoopStart = tCore.SpeedLoopStart;
    ENIE();

    tMotor.ActSpeed = Temp >> 1;
}
/*=============================================================*/
```

好，期望目标转速获取到了，下面要获取反馈转速。反馈转速的话，来自于观测器的估算。而估算器估算的话，对应的是电转速。所以呢，我们需要把电转速变为机械转速。电转速跟机械转速的关系呢，就相差一个极对数的关系。机械转速 = 电转速 / 极对数。因为我们电机是 2 对极的，所以就把电转速右移 1 位，就相当于是除以 2 了。那么，目标转速跟反馈转速都获取到了，下面呢，我们就可以进行速度控制了。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void MotorSpeedLoop(void)
{
    if (Motor_Flag.Bits.SpeedLoopEnable & Motor_Flag.Bits.SpeedLoopStart)
    {
        tPid.Kp = 16384;
        tPid.Ki = tPid.Kp >> 14;
        tPid.Kd = tPid.Kp >> 12;
        tPid.sErr = (signed int)((tMotor.AimSpeed << 3) - (tMotor.ActSpeed << 3));
        tPid.sLimit = 32767;
        PidControl(&tPid);

        if (tFoc.Direction == CCW)
        {
            if (tPid.sOut < SET_IQ_MIN)
                tMotor.qSetIq = SET_IQ_MIN;
            else
                tMotor.qSetIq = tPid.sOut;
        }
        else
```

问：为什么要右移？
答：这个参数是调试出来的，下节课我们会讲调试方法和步骤。

问：&应该是两个吧？这两个有什么区别 感觉一个意思？
答：如果不需要使用速度环，就可以 Motor_Flag.Bits.SpeedLoopEnable = 0。 如果速度环使能 Motor_Flag.Bits.SpeedLoopEnable = 1，刚开始还不能立刻有效，需要等到电流闭环稳定，才可以开始速度环，这时候 Motor_Flag.Bits.SpeedLoopStart = 1。

大家看下，根据速度环控制的两个标志位。如果都是 1 的话，就会使能速度控制。这里呢，需要设定控制环的参数，Kp，Ki，Kd 都要设定好。然后呢，还要求解转速误差，同时，还需要设定限幅值。Motor_Flag.Bits.SpeedLoopEnable 速度环使。
能 Motor_Flag.Bits.SpeedLoopStart 速度环允许开始工作。上面这些值设定好了之后呢，就可以调用 PID 控制的函数了。

```
static void MotorSpeedLoop(void)
{
    if (Motor_Flag.Bits.SpeedLoopEnable & Motor_Flag.Bits.SpeedLoopStart)
    {
        tPid.Kp = 16384;
        tPid.Ki = tPid.Kp >> 14;
        tPid.Kd = tPid.Kp >> 12;
        tPid.sErr = (signed int)((tMotor.AimSpeed << 3) - (tMotor.ActSpeed << 3));
        tPid.sLimit = 32767;
        PidControl(&tPid);
```

这个地方，就是调用我们之前分析过的 PID 函数。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
void PidControl(tPID_Type *tPidStruct)
{
    signed int sTemp;
    signed int sTempD;
    UNION_S32 sTemp32;

    tPID_Type tPidVar;
    tPidVar = *tPidStruct;
    sTemp = CalProportion(&tPidVar) + CalIntegration(&tPidVar);
    sTempD = CalDifferential(&tPidVar);
    sTemp32.S32 = (signed long)(sTemp) + (signed long)(sTempD);
    if (sTemp32.S32 > 32767)
        tPidVar.sOut = 32767;
    else if (sTemp32.S32 < -32767)
        tPidVar.sOut = -32767;
    else
        tPidVar.sOut = sTemp + sTempD;

    *tPidStruct = tPidVar;
}
```

这个函数的输出值呢，就会作为电流环的 Iq 给定值。当然，关于 PID 参数的设定呢，我们后面会讲的哈。这里的话，大家先把代码实现方式理解好。

```
    if (tFoc.Direction == CCW)
    {
        if (tPid.sOut < SET_IQ_MIN)
            tMotor.qSetIq = SET_IQ_MIN;
        else
            tMotor.qSetIq = tPid.sOut;
    }
    else
    {
        if (tPid.sOut < SET_IQ_MIN)
            tMotor.qSetIq = -SET_IQ_MIN;
        else
            tMotor.qSetIq = -tPid.sOut;
    }
}
```

然后这里呢，是考虑电机旋转方向不同的情况，给定电流环的给定值也就不一样。CCW 的时候呢，对应的 Iq 是正值，CW 的时候，对应的 Iq 设定值是负值。

问：速度 pi 和电流 pi 都可以用这个函数吧？
答：可以的，但是中断里的程序应尽量简单些，可以考虑用汇编。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    DISIE();
    tMotor.qSetIq = tIm.qIqGiven;
    ENIE();
    tPid.qSumErr32.S32 = 0;
}

DISIE();
tCore.qSetIq = tMotor.qSetIq;
ENIE();
```

然后，这个 else 的话，对应的是速度环没有使能的情况，没有使能的话，就是速度环输出给电流环工作的给定值，就是 IqGiven 。最后，这个变量统一传递给中断中的 tCore.qSetIq 变量。

```
    tCore.SpeedLoopStart = 1;
    if (tIm.qIqGiven < tCore.qSetIq)
        tIm.qIqGiven ++;
    if (tIm.qIqGiven > tCore.qSetIq)
        tIm.qIqGiven --;
}
```

tCore.qSetIq 变量就相当于电流环的设定值，而 tIm.qIqGiven 变量的话就是电流环的目标值。这里，也做了一定的斜坡处理。如果是单电流环工作的话，没有什么斜坡，因为 tCore.qSetIq 就是 tIm.qIqGiven。速度环工作的话，tIm.qIqGiven 是慢慢靠近速度环的输出的。

```
void MotorModule(void)
{
    GetMotorSetSpeed();//
    GetMotorAimSpeed();//
    GetMotorActSpeed();

    MotorSpeedLoop();
```

然后呢，我们把分析的这几个函数调用后，就可以实现速度闭环控制了。没有问题的话，我们再看一下按键切换方向的功能。

问：如果 Us 模长最大值改小了，那只通过调 Iq 可能调不到目标转速吧？

答：这样的话等于限制了最大功率输出，能否达到设定转速，取决于负载大小和实际的目标转速大小。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
static void KeyScanf(void)
{
    if(!PORTCbits.RC5)
    {
        Dodelay();
        if(!PORTCbits.RC5)
        {
            Motor_Flag.Bits.Key2Press = 1;
        }
    }
    else
    {
        if(Motor_Flag.Bits.Key2Press)
        {
            Motor_Flag.Bits.Key2Press = 0;
            if(tFoc.Direction == 1)
            {
                tPid.qSumErr32.S32 = 0;
                Motor_Flag.Bits.ReqOff = 1;
                tPiId.qSumErr32.S32 = 0;
                tPiIq.qSumErr32.S32 = 0;
                tFoc.Direction = 0;
                DIR_PIN = LED_OFF;
            }
```

这里呢，就是一个按键的扫描，判断按键有没有按下。

```
if(Motor_Flag.Bits.Key2Press)
{
    Motor_Flag.Bits.Key2Press = 0;
    if(tFoc.Direction == 1)
    {
        tPid.qSumErr32.S32 = 0;
        Motor_Flag.Bits.ReqOff = 1;
        tPiId.qSumErr32.S32 = 0;
        tPiIq.qSumErr32.S32 = 0;
        tFoc.Direction = 0;
        DIR_PIN = LED_OFF;
    }
    else
    {
        tPid.qSumErr32.S32 = 0;
        tPiId.qSumErr32.S32 = 0;
        tPiIq.qSumErr32.S32 = 0;
        Motor_Flag.Bits.ReqOff = 1;
        tFoc.Direction = 1;
        DIR_PIN = LED_ON;
    }
}
```

如果说切换方向的按键按下了，就切换另一个方向。同时呢，用一颗表示方向的 LED 来指示方向。切换方向的时候，把对应的控制器的积分清掉。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    tPid.qSumErr32.S32 = 0;
    tPiId.qSumErr32.S32 = 0;
    tPiIq.qSumErr32.S32 = 0;
    Motor_Flag.Bits.ReqOff = 1;
    tFoc.Direction = 1;
    DIR_PIN = LED_ON;
}
```

这里呢，也是同理。这样的话，按下按键后，电机先停一下，然后再切换方向运行。做了一个简单的方向切换函数，验证两个方向的旋转。

```
/*=============================================
void MotorModule(void)
{
    GetMotorSetSpeed();//
    GetMotorAimSpeed();//
    GetMotorActSpeed();

    MotorSpeedLoop();

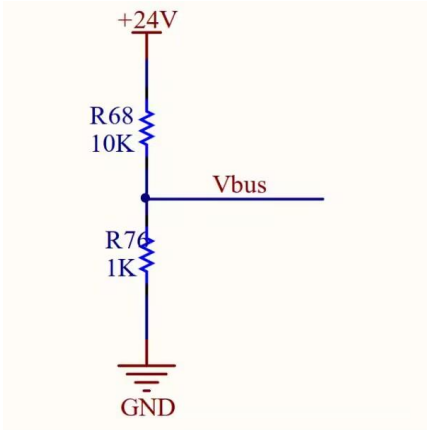
    MotorRunStopControl();

    KeyScanf();
}
```

这个函数呢，也放在主循环中调用。

```
/*============================
void FocApp(void)
{
    AdcModule();
    MotorModule();
}
```

然后，统一放到这个 App 函数中调用。继续啊，速度环控制相关的代码分析完了，再来看一下故障处理。故障的话，咱们这里重点是做了电压及电流的故障检测。电压故障呢，它是基于对 Vbus 电压采样来做的。



这个呢，就是我们的采样电路。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

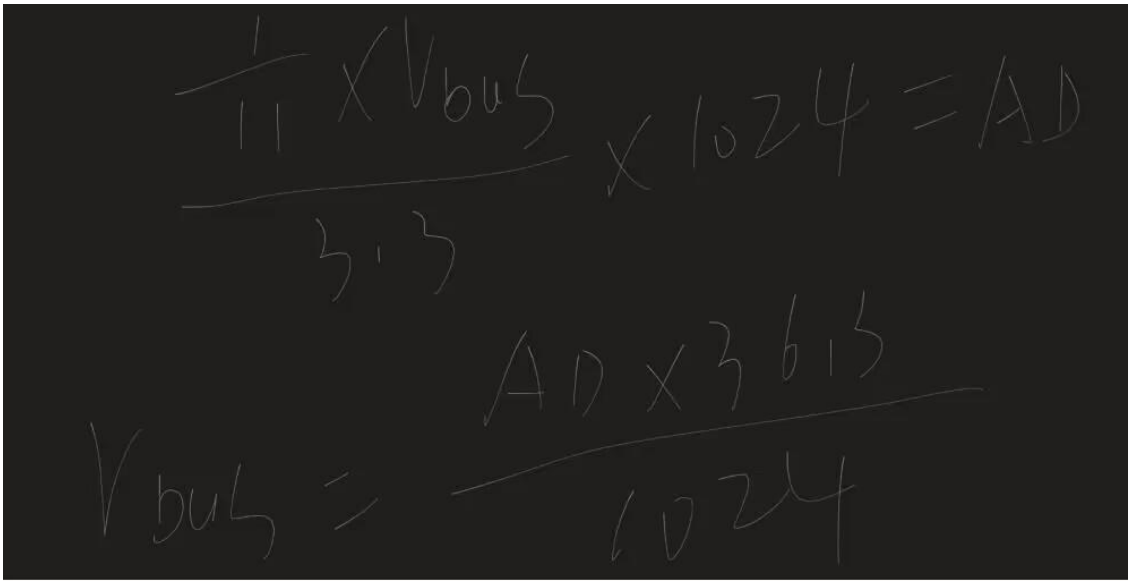
```
/*=============================================================================*/
unsigned int GetVbusVoltage(unsigned int Vbus_Adc)
{
    UNION_U32 unTemp32;

    unTemp32.U32 = __builtin_muluu(VBUS_FACTOR,Vbus_Adc);
    unTemp32.U32 >>= 10;
    return (unTemp32.Words.Low);
}
```

这个函数，它是根据 AD 采样值，来把 Vbus 电压的值给求解出来。

```
/*=============================================*/
#define     VBUS_FACTOR         36
#define     VBUS_LOW_LIMIT      18
#define     VBUS_HIGH_LIMIT     30
/*=============================================*/
```

这个系数呢，是 36，怎么得到的？计算过程？



有上面的公式

```
/*=============================================================================*/
unsigned int GetVbusVoltage(unsigned int Vbus_Adc)
{
    UNION_U32 unTemp32;

    unTemp32.U32 = __builtin_muluu(VBUS_FACTOR,Vbus_Adc);
    unTemp32.U32 >>= 10;
    return (unTemp32.Words.Low);
}
```

这段程序应该明白了。这样的话，可以求解出 Vbus 的值。

```
}
/*=============================================================================*/
void AdcModule(void)
{
    AdcPotProcess();
    tFoc.BusValue = GetVbusVoltage(tAdc.An3.ValueAverage);
}
/*=============================版权归张飞实战电子所有，违者必究！！！=======
```


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这个 BusValue 值就是 Vbus 电压，然后呢，我们可以基于这个值去做电压保护。

```
static void VbusVoltageDetect(void)
{
    if (tFoc.BusValue < VBUS_LOW_LIMIT)
    {
        Error_Flag.Bits.LackVol = 1;
        Error_Flag.Bits.OverVol = 0;
    }
    else if (tFoc.BusValue > VBUS_HIGH_LIMIT)
    {
        Error_Flag.Bits.LackVol = 0;
        Error_Flag.Bits.OverVol = 1;
    }
    else
    {
        Error_Flag.Bits.LackVol = 0;
        Error_Flag.Bits.OverVol = 0;
    }
}
```

大家看下这个函数，就是判断电压是不是小于欠压阈值，还是大于过压阈值。如果说满足欠压的条件，就把欠压的标志位给置 1。同理，满足过压的条件，就把过压的标志位给置 1。这里要注意的一点是，电压值需要经过滤波，瞬时值不能直接拿过来用。根据欠压或者过压的标志位，可以做相应的保护。

```
static void ErrorProcess(void)
{
    if (Error_Flag.Word == 0)
    {
        tProtect.Code = 0;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Motor_Flag.Bits.ReqOff = 1;

        else if (Error_Flag.Bits.OverVol)
            tProtect.Code = 1;

        else if (Error_Flag.Bits.LackVol)
            tProtect.Code = 2;

        else if(Error_Flag.Bits.OverCurrent)
            tProtect.Code = 3;

        else
            tProtect.Code = 0;
    }
}
```

大家可以看下这个故障处理的函数。如果说没有故障的话，故障标志位都是 0，那么对应的 Error_Flag.Word 就等于 0。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
/*==========================================================
typedef union
{
    unsigned int Word;
    struct
    {
        unsigned Hall:1;
        unsigned OverVol:1;
        unsigned LackVol:1;
        unsigned OverCurrent:1;
        unsigned Unused:12;
    }Bits;
}UNION_ERRORFLAGS;
```

大家看下，这里呢是用联合体嵌套结构体的方式定义的。也就是下面的故障标志位，是属于 Word 中的某一位，这样定义的好处，就是可以整体操作也可以位操作。

```
static void ErrorProcess(void)
{
    if (Error_Flag.Word == 0)
    {
        tProtect.Code = 0;
    }
    else
    {
        if (Motor_Flag.Bits.Status)
            Motor_Flag.Bits.ReqOff = 1;

        else if (Error_Flag.Bits.OverVol)
            tProtect.Code = 1;

        else if (Error_Flag.Bits.LackVol)
            tProtect.Code = 2;

        else if(Error_Flag.Bits.OverCurrent)
            tProtect.Code = 3;

        else
            tProtect.Code = 0;
    }
}
```

如果说，出现了故障，就会进入 else 分支。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
else
{
    if (Motor_Flag.Bits.Status)
        Motor_Flag.Bits.ReqOff = 1;

    else if (Error_Flag.Bits.OverVol)
        tProtect.Code = 1;

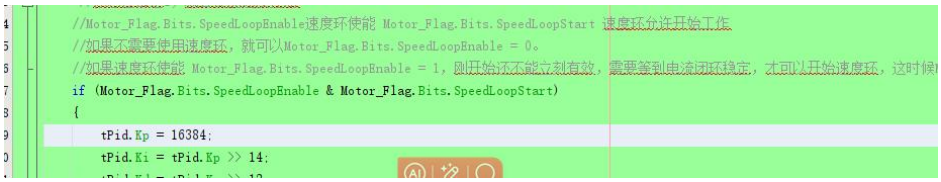
    else if (Error_Flag.Bits.LackVol)
        tProtect.Code = 2;

    else if(Error_Flag.Bits.OverCurrent)
        tProtect.Code = 3;

    else
        tProtect.Code = 0;
}
```

如果说，电机在运行的话，那么就会发出一个停机请求。并且呢，会根据故障的标志位，来给定对应的故障代码。例如，过压故障是 1，欠压故障是 2，过流故障是 3。根据故障代码呢，可以让故障指示的 LED 灯来闪烁对应的次数。这样的话，我们看到闪烁几次，就清楚发生了什么故障。

今天的内容就到这里，下节课我们从分析过流故障开始。

问： 
```
//Motor_Flag.Bits.SpeedLoopEnable速度环使能 Motor_Flag.Bits.SpeedLoopStart 速度环允许开始工作
//如果不需要使用速度环，就可以Motor_Flag.Bits.SpeedLoopEnable = 0。
//如果速度环使能 Motor_Flag.Bits.SpeedLoopEnable = 1，刚开始还不能立刻有效，需要等到电流闭环稳定，才可以开始速度环，这时候
if (Motor_Flag.Bits.SpeedLoopEnable & Motor_Flag.Bits.SpeedLoopStart)
{
    tPid.Kp = 16384;
    tPid.Ki = tPid.Kp >> 14;
```
，这个为什么是位与？

答：需要两个同时成立为真就是位与，有一个不成立就是假。&&用于两个表达式同时成立判断。

大家早点休息，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。